

CarDriVR: Developing an Immersive Modular Virtual Reality Driving Simulator for Consumer-Level Hardware

Joel Boucher

Bachelor of Science in Computer Science
The University of Bath
May 2023

This dissertation may be made available for consultation within the University Library and may be photocopied or lent to other libraries for the purposes of consultation.

CarDriVR: Developing an Immersive Modular Virtual Reality Driving Simulator for Consumer-Level Hardware

Submitted by: Joel Boucher

COPYRIGHT

Attention is drawn to the fact that copyright of this dissertation rests with its author. The Intellectual Property Rights of the products produced as part of the project belong to the author unless otherwise specified below, in accordance with the University of Bath's policy on intellectual property (see https://www.bath.ac.uk/publications/university-ordinances/attachments/Ordinances_1_October_2020.pdf).

This copy of the dissertation has been supplied on condition that anyone who consults it is understood to recognise that its copyright rests with its author and that no quotation from the dissertation and no information derived from it may be published without the prior written consent of the author.

DECLARATION

This dissertation is submitted to the University of Bath in accordance with the requirements of the degree of Bachelor of Science in the Department of Computer Science. No portion of the work in this dissertation has been submitted in support of an application for any other degree or qualification of this or any other university or institution of learning. Except where specifically acknowledged, it is the work of the author.

CONTENTS

Contents	3
List of Figures	4
Abstract	6
1 Introduction	6
2 Related work	7
2.1 Summary of related work	8
3 Design and Implementation	8
3.1 Hardware setup	8
3.1.1 Control mapping	9
3.2 Driving Physics	9
3.3 Driver Model	9
3.3.1 Waypoint system	10
3.3.2 Traffic following	11
3.3.3 Self-driving mode	11
3.4 City generation	11
3.4.1 Traffic lights	12
3.4.2 Pedestrians	12
3.4.3 Tile setup	13
3.5 Motorway generation	13
3.5.1 Motorway future work	13
3.6 Main menu	13
4 Self-driving Experiment	14
4.1 Integration with CarDriVR	14
4.2 Results	15
5 Discussion	15
5.1 Can we design a modular and cost-effective driving simulator for VR? (RQ1)	15
5.2 Can we utilise modern consumer-level VR hardware to simulate realistic driving scenarios? (RQ2)	15
5.3 Future work	15
5.3.1 Infinite city	15
5.3.2 AI traffic improvements	15
5.3.3 Traffic lights	16
5.3.4 Pedestrians	16
5.3.5 Motorway improvements	16
5.3.6 Different vehicle types	16
5.3.7 Cyclists	16
5.3.8 Motion platform	16
5.3.9 Weather and time	16
5.3.10 GPS navigation	16
5.3.11 Eye tracking	16
5.3.12 Additional steering wheel controls	16
6 Conclusion	16
6.1 Word Count	16
Bibliography	16

LIST OF FIGURES

1	Top Left: A user driving a car in CarDriVR. Top Right: A view of the AI traffic in the centre of a city. Bottom Left: Inside view of the car used in the validation study. Bottom Right: Wide view of a simulated city.	6
2	Driving cockpit with Thrustmaster wheel, pedals, and gear shifter	8
3	Turning radius	10
5	Waypoints at a junction	10
4	Top-down view of the city tiles	11
6	A beach tile at the edge of the city	12
7	NavMesh shown in blue on the pavement around a block	13
8	The motorway from the driver's point of view	13
9	Main menu user interface	14
10	Main menu city preview	14
11	Top-down view of the city generated from Figure 10	14
12	The two levels of anthropomorphism used in the self-driving car study	15

Acknowledgements

Firstly, I want to thank my supervisor, Crescent Jicol, for his support and enthusiasm throughout the year. I thoroughly enjoyed this project from start to finish, and it has been the highlight of my time at University.

I also want to thank my supervision group members for keeping me company through countless hours of work, including many laughs despite how stressed we were. In particular, thanks to Lauren, Patrycja, and Sam for basically living in this lab with me for the last two weeks.

Thank you to Aron of NWH Coding, who was constantly on hand for support with the vehicle physics, and who shows great dedication to his work.

Finally, thanks to my parents for proofreading and always being there for me when I needed support.

CarDriVR: Developing an Immersive Modular Virtual Reality Driving Simulator for Consumer-Level Hardware

Joel Boucher
jb2864@bath.ac.uk
University of Bath
Bath, Somerset, UK



Figure 1: Top Left: A user driving a car in CarDriVR. Top Right: A view of the AI traffic in the centre of a city. Bottom Left: Inside view of the car used in the validation study. Bottom Right: Wide view of a simulated city.

ABSTRACT

Driving simulators provide a safe and sustainable environment to study driver experience. Virtual reality is ideal for this application since it allows users to be immersed in a virtual world with accurate scale and depth perception. Previously, technical limitations have compromised the realism of driving simulators and state-of-the-art hardware has been prohibitively expensive for smaller companies or researchers. However, modern VR and graphical processing technology has progressed to the point where it is now possible to bring driving simulators to consumer-level hardware while maximising immersion and realism. This project introduces CarDriVR - a highly customisable VR driving simulator, utilising realistic 3D assets to create large-scale driving environments. CarDriVR is capable of generating cities and motorways with fully functional AI traffic and pedestrians. Additionally, the paper validates an instantiation of such an environment through a comprehensive user study.

1 INTRODUCTION

Understanding drivers' cognitive processes and decision-making while on the road is a challenging task due to the large amount of split-second decisions constantly being made, as well as the need to react to various stimuli through vision, hearing, and tactile feedback. Human error contributes significantly to vehicular accidents, leading to a range of consequences such as fatalities, traffic congestion, supply chain disruptions, travel time unreliability, increased noise pollution, and air quality deterioration [4]. Driving simulators offer a safe and sustainable alternative to explore drivers' reactions to various scenarios without endangering the public.

Historically, achieving a realistic driving experience has required prohibitively expensive technology [2]. However, recent advances in virtual reality (VR) technology have led to affordable head-mounted displays that can deliver a highly immersive experience [22], with large field-of-view angles [44], minimal screen-door effects [44],

and high refresh rates [15]. The combination of these advancements helps to minimize simulation sickness, which is a common discomfort among VR users [15]. This, combined with the fact that modern graphics cards are capable of rendering highly detailed environments, means that now is the perfect time to create a technologically up-to-date driving simulator.

This project proposes CarDriVR - a realistic, cost-effective virtual reality driving simulator using modern consumer-level hardware, facilitating research on existing or new experiences of road users. The simulator will serve as a platform for the development of various road scenarios and driving experiences.

Many driving simulators have been developed over the last few decades by companies and researchers, but often with compromises between realism and affordability. Vehicle manufacturers, in particular, have invested millions in cutting-edge equipment to develop highly realistic simulators, such as the Fiat Chrysler Automobiles Vehicle Dynamics Simulator [2], SHERPA at PSA Peugeot-Citroën [21], and Ford's VIRTTEX [1]. The technology used in these simulators is currently unobtainable or unachievable for smaller scale enterprises or academic research centres.

In consequence, this project aims to answer the following research questions:

RQ1 Can we design a modular and cost-effective driving simulator for VR?

RQ2 Can we utilise modern consumer-level VR hardware to simulate realistic driving scenarios?

To answer these questions, a thorough examination of related work was conducted, paving the way for the development of the proposed driving simulator. Secondly, the resultant simulator - CarDriVR - was tested in a comprehensive lab-based user study investigating induced presence and perceived realism. The results presented validate the use of modern consumer-level hardware and modular software design as a viable alternative to the traditionally limited and cost-prohibitive options.

2 RELATED WORK

Driving studies on real roads have been performed historically by both vehicle manufacturers and researchers. In particular, manufacturers used real road tests to develop and refine vehicle performance, while researchers are more interested in human responses to driving stimuli. However, they can be expensive, time-consuming, bad for the environment, and unpredictable due to factors such as road and weather conditions. Over time, companies, and academics have transitioned to using driving simulators to perform these critical evaluations. Driving simulators provide a safe environment to create repeatable scenarios, allowing engineers to quickly iterate on different designs, or researchers to modify specific individual conditions.

One issue facing smaller scale enterprises or academic research centres is that the technology required to produce the most realistic experiences is currently unobtainable or unachievable with smaller budgets. Mohajer et al. [25] performed a survey of some well-known expensive driving simulators that feature motion bases with multiple degrees of freedom. These include the National Advanced Driving Simulator (NADS) in Iowa, USA [13], Daimler-Benz Driving Simulator, Germany [9], SHERPA at PSA Peugeot-Citroën [21], and

Ford's VIRTTEX [1]. Mohajer et al. [25] classified these simulators using two key criteria, fidelity, and usability. Fidelity refers to the quality in which the simulator replicates the vehicle capabilities faithfully. Usability describes how versatile and reconfigurable the simulator is for various studies. For research purposes, high levels of fidelity and usability are required. The goal of CarDriVR is to produce high levels of fidelity and usability in a virtual reality driving simulator, without the previously restrictive cost.

To achieve a high level of fidelity, the handling physics of the vehicle must be realistic. Modern consumer-grade gaming hardware can be harnessed to provide feedback to the driver about the state of the vehicle. Gaming hardware companies like Logitech, Thrustmaster, and Fanatec sell a wide range of driving simulator equipment such as steering wheels, pedals, and gear shifters [10, 24, 36]. The development of this hardware has in part been a result of the surge in popularity of sim racing during the COVID-19 pandemic [37]. This equipment is capable of providing deeper levels of immersion to drivers through force feedback [26]. Force-feedback is created by a motor within the steering wheel, which can vibrate or resist the drivers' actions to make them feel like they're really controlling a vehicle. Mohajer et al. [25] states that having accurate vibration in the steering wheel which mimics the tire-road interaction provides the driver with cues for their current speed and trajectory.

The software used for large scale driving simulators is often highly bespoke, designed specifically for a single dedicated simulator. There is a gap in the driving simulator space for a simulator that is much more flexible and can run on consumer-level hardware. Video games are a possible alternative due to their low cost to the researcher, graphical realism, and accurate vehicle physics. de Frutos and Castro [7] looked at the use of video games for low-cost driving simulation research. They used both rFactor 2 and Assetto Corsa to study road geometry, and found that creating the 3D scenarios was a time-consuming task, but provided realistic results. Both simulators were found to have accurate driving physics and provided enough telemetry data to be useful for researchers. However, due to the high amount of time spent creating the scenarios, the usability of video games for research is quite low. The goal of CarDriVR is to create a simulator similar in fidelity to these video games, but with a higher usability, such that it takes much less time to produce the desired environments and scenarios. To achieve this, the simulator will be created using a modular design pattern, meaning it can be easily configured to suit the requirements of a study.

In the case of VR, immersion can be defined as those technical factors that help drive user feelings of presence. To achieve a realistic driving experience, it is therefore important to first examine the immersive factors that make users feel present in a virtual environment. Ihemedu-Steinke et al. [14] found that some essential characteristics are to have an accurate sense of speed and a realistic AI traffic model with other cars and pedestrians in the environment. Olstam [29], Olstam et al. [30], and Felez et al. [11] were identified as a foundation for the AI traffic model to be used in CarDriVR. These papers describe techniques to include a microscopic traffic model in driving simulators.

Research in autonomous urban driving is hindered by infrastructure costs and the logistical difficulties of training and testing

systems in the physical world. Instrumenting and operating even one robotic car requires significant funds and manpower [8]. The researchers at Intel Labs, Toyota Research Institute, and the Computer Vision Center, Barcelona identified the limitations in existing driving simulators and introduced CARLA (Car Learning to Act) – a driving simulator designed primarily for use in training autonomous vehicles. This is a low-cost and high-fidelity solution. However, it does not provide the required level of immersion that can be achieved by using virtual reality.

One of the main focuses of Kemeny [21] was on how to reduce so-called simulation sickness, caused by conflicts in the vestibular-ocular reflex, where the driver sees rotations not perceived by the inner ears. Simulation sickness is a common discomfort experienced by users in virtual environments [32]. This was a particularly important focus for CarDriVR because the future use of the simulator involves studying the effects of different driving conditions on users, and the results must not be compromised by the participant feeling ill. Cao et al. [3] found that participants were able to drive in their driving simulator in VR for around 50 minutes before feeling simulator sickness. To achieve this, they recommend steps towards reducing simulation sickness for participants: In the case of driving simulation, look around the scene first, and then start driving slowly. Avoid sharp turns in the beginning, and gradually increase driving speed. Take short breaks after playing for about 5 min, and gradually increase play duration.

2.1 Summary of related work

Ihemedu-Steinke et al. [14] showed that an essential component to making the simulator feel realistic was to have AI traffic in the environment, including other vehicles on the road, and pedestrians walking on the pavements. Users of the driving simulator will have to adapt their driving to account for the actions of other road users. Both the user controlled car and AI vehicles will drive using a fully simulated physics-based system, meaning they will also accelerate, brake, and handle like a real car.

Another essential characteristic of a driving simulator was determined to be a realistic sense of speed. This sense of speed will be created firstly by using an accurate 1:1 scale for the environment, including vehicles, roads, and buildings. Another factor which affects the sense of speed in virtual reality is the field of view angle [31]. To achieve a more immersive experience, the VR headset will be chosen based on its field of view, aiming to maximise the driver's perception of the environment.

3 DESIGN AND IMPLEMENTATION

Based on the related work, the scope of this project was determined to involve creating a virtual environment in which a user can experience driving a vehicle from a first-person perspective. It was designed using a modular pattern, meaning that it can be easily adapted for different scenarios. The hardware used for this project was all off-the-shelf gaming equipment, which was carefully chosen and assembled to achieve a balance of cost-effectiveness and ability to produce a realistic simulation.

CarDriVR was made in the Unity game engine, due to its compatibility with a wide range of VR devices and input devices such as steering wheels, its built-in physics simulation capabilities, and

large community support. The project made use of multiple assets from both the Unity asset store and turbosquid. These assets range from a vehicle physics solution [5], to 3D models of vehicles [23] and cities [17–20].

3.1 Hardware setup

The input devices for the driving simulator in this project were a Thrustmaster T300RS GT steering wheel and Thrustmaster TH8A gear shifter. The wheel features force-feedback, which is used to give the driver a realistic steering experience, by replicating the forces that would be felt if they were behind the wheel of an actual car [16]. The simulator calculates the desired sensations of resistance and vibration based on the road surface and tyre physics. The wheel was supplied with a set of three pedals, which are used alongside the gear shifter to accurately represent driving a manual car. The wheel also has paddle shifters, which are utilised to control cars without a manual gearbox.

The wheel, pedals, and shifter were mounted to a Next Level Racing GTtrack simulator cockpit [27]. Using a cockpit allows drivers to sit in a more natural position and operate the controls in a way that closely mimics driving a real car. Since the simulator is exclusively in virtual reality, there was no need to try to make the cockpit look like a real car, as previous studies have done [2], since users will only be interacting with it through touch.



Figure 2: Driving cockpit with Thrustmaster wheel, pedals, and gear shifter

The virtual reality headset was a Valve Index, which provides a wide field of view of 130 degrees, an image resolution of 1440 × 1600 per eye, and a refresh rate of 144 Hz. Higher frame rates reduce the time between head movement and visual updates and “improve realism and optical comfort [42].” This headset was chosen as it

provides one of the widest fields of view available within consumer-grade head-mounted displays. Field of view has been shown to affect speed perception, partly due to the role of peripheral vision in motion perception by the human visual system and partly because of the corresponding generated immersion level [21, 31]. A wider field of view means that the user feels a more realistic sense of speed, and is more immersed in the simulator.

The PC used to run the simulator had the following specifications:

Processor (CPU) Intel® Core™ i9 24-Core Processor i9-13900K

Graphics Card 24 GB NVIDIA GeForce RTX 4090

Memory (RAM) 64 GB Corsair DOMINATOR PLATINUM RGB DDR5 5200MHz

Having a high-end PC for the simulator means that more of the environment can be rendered, while maintaining a high and stable frame rate required for virtual reality applications.

3.1.1 Control mapping. Various vehicle controls had to be mapped to physical buttons on the steering wheel. Using the Unity input system, most of the common controls for a real car were included, allowing users to feel close to their real experiences of driving. These include indicators, horn, and headlights, along with other controls for the simulator. However, since the T300RS GT doesn't include stalks next to the wheel, controls had to be mapped to buttons on the front of the steering wheel. Indicating left and right had to be mapped to the *L2* and *R2* buttons, respectively. When finding a steering wheel to buy for this project, none of the current commercially available options featured stalks, so other features such as force feedback were prioritised.

When sitting in the cockpit and putting on the VR headset, it was important that the driver felt as though the virtual car was in the correct place, and the locations of the virtual seat and steering wheel matched exactly with the locations of the real seat and steering wheel. Since the cockpit and the VR base stations can be moved around, the position in the simulation doesn't always line up with the real position. To solve this, a function was added which moves the position of the VR camera to the predefined driver position within the car. Firstly, the camera is rotated about the vertical axis, so the driver is looking straight ahead towards the front windscreen. Then the position is moved to the centre of the seat at eye level. It is important to note that the rotation in the *X* and *Z* axes is not changed, as this would tilt the world relative to the flat surface of the real world, and could cause the user to experience increased simulation sickness.

3.2 Driving Physics

CarDriVR was developed using a vehicle physics asset, NWH Vehicle Physics 2 (NVP2) [5], which was purchased from the Unity asset store. This asset was used for both the user controlled vehicle and the AI traffic vehicles.

There were many advantages to using an existing asset, along with saving precious development time. They have a proven reliability, since they have been tested and refined over time, resulting in stable and reliable performance. This was particularly crucial in the VR application, since any instability or lack of realism in the vehicle physics would cause a user to lose their sense of presence in the simulation, and it could also lead to an increase in simulation

sickness. By selecting a proven asset, there could be confidence in the quality of the driving experience.

Another advantage to using NVP2 was that it has a modular design. This means that it was designed to be easily adapted and integrated into various projects, which was ideal for this driving simulator. This modularity enabled the customisation of the physics of the AI traffic to increase performance when many vehicles were present in the scene, which was essential for running the simulator on consumer-level hardware. It also meant that different types of vehicles could be created, such as sedans, SUVs, and sports cars. Each of these had different driving characteristics, from acceleration and handling to the sound of their engines.

Since the asset is a full vehicle physics package, it came with various tools that make vehicles seem more realistic. For example, the asset features a dynamic sound system, which meant that both the user controlled car and the AI traffic were able to produce realistic engine, turbo, transmission, suspension, tyre, brake, collision, indicator, and horn noises. There are also particle systems which create smoke out of the car's exhaust and skid marks on the road. NVP2 also provides a dashboard similar to a real car, which shows the current state of the vehicle through a speedometer, RPM gauge, and engine warning lights.

The asset also has built-in compatibility with a wide variety of input devices. In particular, it is compatible with the Thrustmaster wheel and gearstick and can provide realistic force feedback. However, while it is compatible with the Thrustmaster equipment, the developer focused mainly on the Logitech ecosystem of steering wheels. Logitech wheels combine the gearstick inputs with the buttons on the steering wheel and communicate all of these to the PC using a single USB cable. However, the Thrustmaster gearstick connects to the PC with a separate USB cable to the steering wheel and pedals, so it was necessary to edit the asset's `SteeringWheelInputProvider` script to use inputs from two separate devices.

Finally, the asset comes with the peace of mind of being able to contact and get support from the developer, as well as frequent ongoing updates and improvements. Even in the short amount of time this driving simulator has been in development, these updates have improved both the performance and realism of the vehicle physics [6].

3.3 Driver Model

A large part of the project was to implement the AI traffic system. This was essential, since it has been shown to be a key factor in creating the sense of realism when driving around in a simulator [14].

The driver model in Section 5 of Felez et al. [11] has been implemented to provide a realistic representation of actions exerted by drivers on the vehicles in the traffic simulation. The driver model determines the driving behaviour based on the surrounding traffic conditions and the driver's individual characteristics, such as their driving style (passive, moderate, or aggressive).

The implemented driver model is an evolution of the vehicle-following theory, which is designed to make each vehicle follow a predetermined path while adapting to the traffic conditions. To achieve this, proportional-derivative (PD) controllers are employed

that act on the vehicle's steering, accelerator and brake. These PD controllers allow the driver model to exhibit various manoeuvres commonly observed in urban traffic circulation, such as lane changes, and behaviours at crossings and intersections.

The decision-making criteria within the driver model dictate the actions the model must take to accelerate, brake, or change lanes, depending on the traffic and signposting conditions at any given moment. This ensures that the driver model exhibits a realistic driving behaviour that responds appropriately to its environment.

It is important to note that while the driver model from the original paper has been implemented, the vehicle model and traffic light model used in this project are different. The vehicle model is handled by the vehicle physics asset, and the traffic light model is based on the UK traffic light system [34]. Section 3.4.1 discusses the traffic light model in more detail.

3.3.1 Waypoint system. To create the predetermined path that the driver model will follow, a waypoint system was implemented. This involves placing multiple "waypoints" along the road, and connecting them in a doubly linked list structure. The waypoint system shown in Game Dev Guide [12] was used and adapted to suit vehicles in the driving simulator. Specifically, the width of waypoints had to be increased to the size of a road lane, and a speed limit and trigger collider were added.

At all times, the driver model has a waypoint as its current target, and due to the doubly linked list structure, it can retrieve the waypoints both before and after the target. It uses these along with data about the vehicle's current speed to determine a reference point towards which it will try to turn. As the speed of the vehicle increases, the reference point gets further along the road, so the steering angle required is shallower. Consequently, at slow speeds, the car is better able to manoeuvre around tight corners.

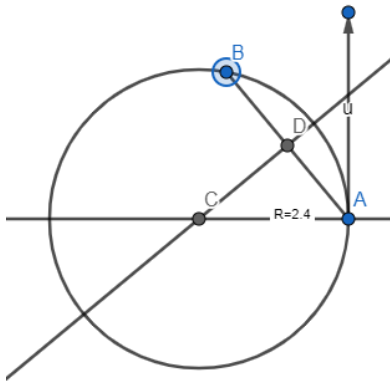


Figure 3: Turning radius

Figure 3 shows the points involved with calculating the turning radius towards the reference point, where

- A is the location of the controlled vehicle
- B is the reference point on the path
- C is the centre of rotation
- D is the point halfway between A and B
- $\vec{u}$ is the velocity vector of the controlled vehicle

- R is the turning radius
- $\vec{AC}$ is perpendicular to $\vec{u}$
- $\vec{DC}$ is perpendicular to $\vec{AB}$

Since the driving simulator is 3-dimensional, the point B could be at a different elevation to point A . Therefore, the vectors were flattened by taking only their X and Z coordinates, turning it back into a 2-dimensional problem. For the turning radius calculation, a function was written which takes A , B , and $\vec{u}$ as inputs, and returns R . Calculating D is trivial as it is simply $\frac{A+B}{2}$. The direction of vectors $\vec{AC}$ and $\vec{DC}$ can be calculated using Unity's Vector2.Perpendicular function. The point C is at the intersection between those vectors. Radius R can be calculated using the following equation.

$$R = \frac{DC_x * (A_y - D_y) - DC_y * (A_x - D_x)}{AC_x * DC_y - DC_x * AC_y} \quad (1)$$

A waypoint also stores more information about the road condition to increase the adaptability of the AI traffic. For example, each waypoint stores a speed limit for that section of road. This allows the traffic to speed up or slow down based on the type of road. Most of the waypoints in the city are set to a speed limit of 20 mph, while waypoints on a motorway section are set to 70 mph. Waypoints also store a friction coefficient, which is used by the driver model to limit the acceleration based on a desired circulation speed, determined by the road conditions (maximum speed limit, radius of curvature, etc.) [11]. The benefit of this system is that it can be used to create more varied driving environments in the future, such as countryside roads or smart motorways.

Waypoints also have a defined width, which represents the size of the lane. This width is used to create a box collider trigger in the area above the waypoint, so that when a vehicle drives into the box, it can set the next waypoint as its target.

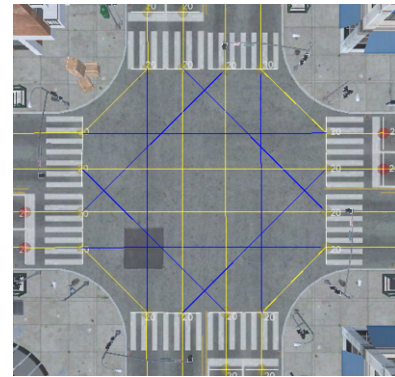


Figure 5: Waypoints at a junction

To handle junctions, a waypoint can optionally store a list of branches. These branches are other waypoints on different roads, which creates a diverging path system. Depending on the scenario, the driver model can either choose a branch randomly or follow a predetermined route. Figure 5 shows the waypoints at a junction. The yellow lines indicate the default path, while the blue lines indicate the branches. In the city environment, the cars drive on the right-hand side of the road, and there are two lanes for each



Figure 4: Top-down view of the city tiles

direction of travel. The right-hand lane can turn right or go straight ahead, and the left-hand lane can go straight ahead or turn left.

3.3.2 Traffic following. Each waypoint contains a list of vehicles which have that waypoint as their current target. This is useful since it allows a vehicle access to the position and velocity data of the vehicle in front, and can then slow down or speed up to maintain a safe distance to the car ahead. To implement overtaking or changing lanes, vehicles needed to be removed or added in the middle of the list, so the list was created as a C# List. This allows for vehicles to be accessed, added, and removed by index.

The minimum safety distance to the vehicle ahead was calculated using Equation 2, which is a modified version of equation 14 from Felez et al. [11]. In particular, the length of the other vehicle was no longer necessary, since the distance is measured between the front of the controlled vehicle and the rear of the other vehicle.

$$d_{s-ij} = l_i \left(0.7 - \frac{c_{ai}}{c_{a_{max}}} \cdot 2 \right) + d_{min} + \frac{u_i^2 - u_j^2}{2a_d} \cdot \frac{c_{a_{max}}}{c_{ai}} \quad (2)$$

Where

d_{s-ij} is the minimum distance between vehicle i and vehicle j

l_i is the length of the controlled vehicle

c_{ai} is the aggressiveness coefficient of the controlled vehicle (value 1 to 3)

$c_{a_{max}}$ is the maximum coefficient of aggressiveness (value = 3)

d_{min} is the minimum distance between stopped vehicles in the city ($d_{min} = 0.5$ m)

a_{di} is the acceleration according to degree of aggressiveness of the controlled vehicle

u_i is the speed of controlled vehicle

In the original paper, this equation is used to determine if a car in another lane is too close to perform a lane change. However, in this project, the safety distance was used to determine the minimum distance to the car ahead. If the car ahead slows down due to traffic signals or other vehicles on the road, the current car will get closer. When the distance is below the safety distance, the reference speed will be lowered to increase the distance again. Since the safety distance is proportional to the current speed of the vehicle, it will maintain a bigger gap at high speeds. This accounts for the increased stopping distance due to the combination of both increased thinking and braking distances [35].

3.3.3 Self-driving mode. One of the benefits of using the same vehicle physics for the autonomous traffic cars and the player-controlled car was that the driver model script could be added to the player car. This means that the car can be switched into a fully automated self-driving mode, essentially turning it into one of the traffic cars, while still containing the virtual reality camera. The self-driving mode was extended to include a fully predetermined route that could be easily adjusted using the main menu – detailed in section 3.6 – and was used in a study on self-driving cars discussed in detail in section 4.

3.4 City generation

The city environment was created using existing 3D model assets [17–20]. The models that were used are in a modular tile format, with each tile being 100×100 metres in size. One tile represents one city block, with two road lanes on each side of the tile. The benefit to using this modular format is that the tiles can be easily combined by mixing them, rotating them, and placing them in a

grid, allowing for many possible city layouts. An example of an instantiation of the city tiles can be seen in Figure 4.

A city manager script was created to handle the randomly generated city. It accepts multiple parameters to modify the layout of the city, such as the height and width of the grid, which tile prefabs to use, whether to create a border around the city, and what percentage of the city should be suburban. The border around the city was originally a small wall to stop vehicles from falling off the edge. However, this was later improved by the utilisation of the kasiopy [17] asset which has a 200-metre stretch of beach, along with two suburban and two city style tiles. The border was changed from the wall to the beach, which creates the sense that the city is on an isolated island in the ocean, rather than floating in space, and thus more realistic. The beach can be seen in Figure 6.



Figure 6: A beach tile at the edge of the city

3.4.1 Traffic lights. The traffic light system used a simple but effective method to change states. The tiles of the city included appropriate traffic lights at each junction, but since they were just a 3D model, all three lights are on by default. To implement the traffic light system, a traffic light prefab was created which contained three black boxes aligned vertically. This prefab was placed in front of each traffic light, making it seem as though all three lights were off. A traffic light manager script was written which makes certain boxes invisible. Making a box invisible creates the illusion that the corresponding light has been turned on. The traffic lights use the UK traffic light convention [34], with the following four possible states that are cycled through by the script.

- Stop** Red light
- Get ready** Red light and Amber light
- Go** Green light
- Stop if safe** Amber light

Some traffic lights can be grouped as they portray the same information to drivers. These include traffic lights that have been placed at different heights so that different vehicles can see them, depending on the type of vehicle or location relative to the lights. In the city scene, the traffic lights on roads going north/south will be linked, while the traffic lights on roads going east/west will also be linked, but on the opposite side of the cycle.

The grouping of traffic lights was inspired by section 6 of Felez et al. [11], which designates a master traffic light from which to base the states of all the other traffic lights at a junction. However, because of the nature of the city layout, at the edges of the city, some traffic lights do not exist. This made it inconsistent as to which tile should contain the master traffic light at any given junction.

The final implementation has a master traffic light for every corner of every tile. The corners of every tile have one traffic light at the entrance to the junction and two at the exit. The entrance traffic light is assigned the master role and controls the other two. Since they are in different directions, the master is always in the opposite phase to the other two. To synchronise the lights at a junction across tiles, the same timing sequence was used across the entire city. To achieve this, a minimum and maximum time can be specified in the traffic light script, and during the initialisation of the simulator, a sequence of green light times is generated using random numbers between these extrema. This gives variety to the timing of the lights, while allowing the traffic lights to stay in sync.

Now that the traffic lights can change state and are synchronised across a junction, the AI traffic needs to be able to react to the lights and stop accordingly. To achieve this, each traffic light contains a list of waypoints on the stop line the traffic light represents. When the traffic light is in any state besides *Go*, the waypoint “stop” variable is set to true. The driver model checks whether its current waypoint is set to stop, and if so, calculates the safety distance using a modified version of equation 2 - shown below in equation 3 - which assumes the vehicle ahead has a speed of 0 and l_i does not scale with the aggressiveness of the driver.

$$d_{s-ij} = 0.3l_i + d_{min} + \frac{u_i^2}{2a_d} \cdot \frac{c_{amax}}{c_{ai}} \quad (3)$$

If the vehicle is within the safety distance, the reference speed is set to 0 and the vehicle will start to brake. The slowing of the vehicle causes the safety distance to decrease, so the vehicle is longer within the safety distance and the reference speed is increased again. This goes back and forth until the vehicle has stopped next to the line. Stopping means that $u_i = 0$, and so $d_{s-ij} = 0.3l_i + d_{min}$.

3.4.2 Pedestrians. Another step towards realism in the driving simulator was to have pedestrians walking around. The asset used to generate pedestrians in CarDriVR is Unity Multipurpose Avatar [38]. This asset provides human avatars with a large amount of customisation, meaning that every pedestrian looks different and wears different clothes, just like in real life. The pedestrians navigate the city using the NavMeshAgent component [40], which provides an object with the ability to navigate a NavMesh. A NavMesh is a data structure in Unity which describes the walkable surfaces in the world and allows agents to find a path from one walkable location to another [39]. The NavMesh was created such that the pedestrians could only walk on the pavement around the tiles, and not into the path of vehicles on the road. To achieve this, a plane was placed on the pavement of each tile, and the experimental NavMeshSurface component [41] was added to each plane to turn them into a NavMesh.

A pedestrian generator script was used to create the pedestrians and distribute them evenly across the city. Along with the



Figure 7: NavMesh shown in blue on the pavement around a block

NavMeshSurface, each tile has a list of pedestrian waypoints at the corners of the block. The pedestrian generator creates a user-specified number of pedestrians per block and spreads them across the waypoints. Finally, a simple pedestrian movement script sets the current target of the NavMeshAgent to a waypoint, and changes the target to the next waypoint when the pedestrian gets close enough. This system could easily be adapted for other future environments in different settings, with longer pedestrian paths or different pedestrian densities. In this simulation, the NavMesh was set up so that pedestrians don't walk into the road at all, but in the future, it could be extended to include road crossing points.

3.4.3 Tile setup. When creating the city using tiles, some information was required to be shared between tiles. The waypoint system meant that certain key waypoints needed to be linked between tiles. To join the waypoints as shown in Figure 5, a KeyWaypoints component was added to each tile prefab, which stores the waypoints at the entrance and exit of a junction for each corner of the tile. The city manager script uses these key waypoints along with the orientation of the tile to calculate which waypoints should be joined or added as a branch.

3.5 Motorway generation

Separate from the city environment, progress was made towards a procedurally generated motorway system. The VIS Games [43] asset was purchased, which provides a selection of modular motorway sections. Each section comes with three different styles of central reservation and each with either two or three lanes. These sections are: a straight section, a 15-degree turn, a 30-degree turn, a motorway exit, a straight section with signs in the middle, and a section with parking for services. Placing sections with the same style and number of lanes next to each other creates a continuous motorway. There are also some transition sections to change between styles or number of lanes.

Since the motorway sections can curve and have different lengths, there needed to be a way to record where exactly to place the next section. Similar to the KeyWaypoints component for city tiles, a MotorwayDetails component was used which stores the style, number of lanes, curve, and length of a section. Each section is stored as a Unity prefab and can be instantiated by the MotorwayManager script. During initialisation, the motorway manager iterates through a user-specified number of sections. In each iteration, it randomly selects a prefab from the prefabs list, and checks if it matches the current style and number of lanes. If it matches the criteria, it instantiates the prefab at the calculated location and rotation. Depending on the MotorwayDetails of the section, the position, and rotation of the next section are updated accordingly. The loop continues until the desired number of sections have been placed, generating a long section of motorway in the scene. A first person view of the motorway can be seen in Figure 8.



Figure 8: The motorway from the driver's point of view

3.5.1 Motorway future work. In the future, the motorway could be dynamically generated as the user is driving. Currently, in CarDriVR, you can specify how many sections to place during the initialisation of the scene, but no more sections are ever placed. This means that if a user drives for long enough, they might reach the end of the motorway. Generating new sections of motorway as the user gets near the end means they could have an endless driving experience.

The motorway could also be made more realistic by including AI traffic. Waypoints have been added to each lane of each section of motorway, which the AI traffic could follow similar to the city environment. These can be connected between sections and used in the future for generating new vehicles and investigating interactions between drivers on high-speed, multi-lane roads.

3.6 Main menu

While the configuration of the city and motorway can be manipulated using the Unity inspector, when building the project into a single program, there needed to be a way to change between the two scenes and allow the user to customise them during runtime. To achieve this, a main menu scene was created, which features a user interface for configuring various aspects of the simulation. In the main menu, there is a replica car to the city scene, so the driver can reset their position and get comfortable with the controls in an empty environment before being put into the full simulation. This was a requirement taken from previous work focusing on simulator sickness. Users of CarDriVR can ease into the simulation

to prolong their experience while reducing feelings of nausea and disorientation.

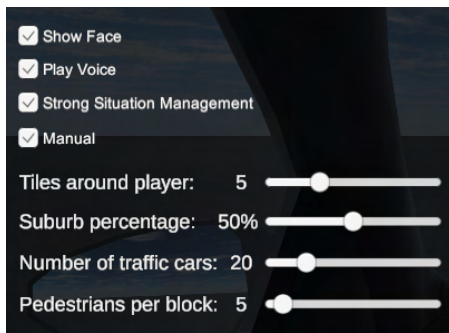


Figure 9: Main menu user interface

As shown in Figure 9, there are options for customising the levels of anthropomorphism and situation management in the experiment, detailed in section 4. There are also settings for traffic density, and pedestrian density, along with the size and layout of the city. There is a preview window, shown in Figure 10, showing the city blocks in the centre with the suburban blocks around the outside. This scales dynamically when the configuration is changed using the size and layout controls. The actual tiles used are chosen randomly from the city and suburban prefab lists, but can be fixed using a random seed. This preview also has the controls to customise the self-driving route taken by the car, shown in light blue. The resulting city from this particular preview can be seen in Figure 11. The beaches are automatically added to surround the city.

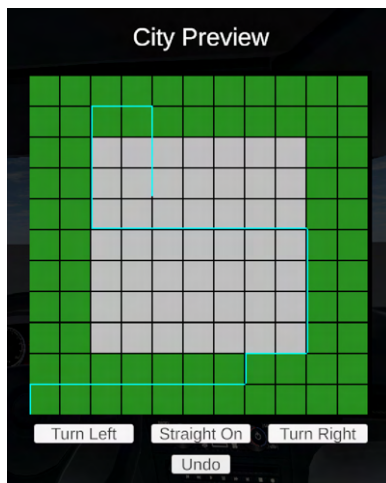


Figure 10: Main menu city preview

The user interface for the main menu can be adapted easily to add more options in the future. The main menu provides a platform for switching between scenes, enabling users to create new modular environments and expanding the possibilities for creating different scenarios.

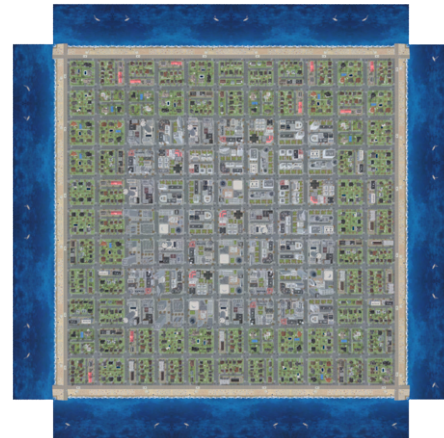


Figure 11: Top-down view of the city generated from Figure 10

4 SELF-DRIVING EXPERIMENT

The driving simulator developed in this project was designed to be able to replicate various road and traffic situations to assess drivers' responses and behaviours in different scenarios. Subsequently, a fellow student, Lauren Pococke, adapted this VR driving simulator to perform a study investigating the effects of varying levels of anthropomorphism and situation management in self-driving cars. Figure 12 demonstrates the two levels of anthropomorphism used in the self-driving car study. This study serves as a validation of the immersive features of the driving simulator. Anthropomorphism refers to the attribution of human characteristics, qualities, or behaviours to non-human objects, such as self-driving cars. Situation management refers to the process of assessing, adapting, and making decisions in response to a dynamic environment. In this case, the vehicle was given a face and a voice and spoke to the participant as it drove itself through the virtual city. The vehicle encountered multiple scripted events, and responded differently based on the situation management condition.

43 participants took part in the study and filled out a survey on presence and realism. In the study, the participants were not controlling the vehicle, so their experience of the simulator is purely a reflection of the environment and its realism.

4.1 Integration with CarDriVR

The study utilised the route planning system on the main menu, which calculates the correct lanes and waypoints to use for the self-driving car. This meant that the car would always take the same route, on which events could be triggered. The index of the current waypoint was used to trigger events when the car got near enough. Such events include pedestrians walking into the road, cars pulling out in front of the self-driving car, and obstacles like traffic cones blocking a lane. For each event, depending on the study condition, the self-driving car would explain to the user how it is handling the situation. This was achieved by using a Unity Audio Source object placed inside the car, and triggering different audio clips based on the condition.



Figure 12: The two levels of anthropomorphism used in the self-driving car study

4.2 Results

Participants felt that the environment was realistic ($M = 3.75$, $SD = 0.67$ on a 5 point scale) and felt physically present in the virtual environment ($M = 3.22$, $SD = 0.83$ on a 5 point scale). The simulator was not motion sickness inducing ($M = 739.79$, $SD = 278.5$ out of a maximum of 2152. Scaled down to a five point scale gives $M = 1.72$, $SD = 0.65$), showing that the combination of the modern consumer-level VR technology with the environment of CarDriVR does not create significant feelings of simulator sickness.

Users judged the safety of the self-driving car to be good ($M = 3.09$, $SD = 1.07$ on a 5 point scale), and accepted the journey ($M = 68.95$, $SD = 22.49$, on a 100 point scale).

When asked about their impressions of the simulator, participants were very positive:

- “It felt really real to me”
- “Good, it felt like I was in a real car, especially in terms of sound”
- “The virtual environment was captivating and the idea of being driven around was appealing to me ... overall it was great.”
- “It was impressive that it felt like you were moving along.”
- “I was very impressed by the system and interface and it has given me hope that self-driving systems will be looked into further as a more realistic and safe possibility in the future.”

5 DISCUSSION

5.1 Can we design a modular and cost-effective driving simulator for VR? (RQ1)

Overall, this project was successful in designing and creating a modular driving simulator for virtual reality. The simulator was cost-effective as it can run on consumer-level hardware and the assets used to create the environment were a low-cost one-off payment. The city tiles and motorway sections, in particular, allow the simulator to generate many possible layouts and scenarios. This demonstrates that the simulator is modular since it can be used to create a wide range of scenarios and the various factors can be configured, such as the controlled vehicle, AI traffic, and pedestrians.

Modularity is further shown by the main menu system, which allows for the addition of new environments in the future. The waypoint system, which governs the paths for AI traffic, provides an easy solution for adding traffic to other scenes.

5.2 Can we utilise modern consumer-level VR hardware to simulate realistic driving scenarios? (RQ2)

The study detailed in section 4 utilised CarDriVR and validated its level of realism and ability to induce immersion in 43 participants. The valve index was used as a representation of the current consumer-grade VR hardware available, and was successful in producing an immersive experience for users.

Participants in the study found that the self-driving car was safe. Since the self-driving car in the study was making decisions based on the same driver model as the AI traffic, this indicates that the traffic is also capable of driving in a safe and realistic way.

5.3 Future work

CarDriVR was fundamentally designed to be used for various future projects and research studies. The current state of the simulator is a representation of the time and resources available in an undergraduate project. As such, there are many features and improvements that could be made to the driving simulator in the future.

5.3.1 Infinite city. The city manager currently is only capable of generating a fixed size grid. It would be possible to change this so that new tiles are generated as the driver enters new areas of the city, effectively creating an endless cityscape.

5.3.2 AI traffic improvements. The driver model which controls the AI traffic could be improved further by adding more awareness at junctions. In this project, the traffic lights are the primary method of reducing the number of collisions between crossing traffic. However, it is still possible that cars turning left and crossing the path of oncoming traffic can collide. This is due to how the vehicles become aware of each other. In this project, they can only react to cars ahead of them in the same lane, and not cars in any other lane. In the future, a system which lets vehicles interact through proximity or even driver field of view could improve the realism of their behaviour significantly, adding actions like lane changes and pausing for oncoming traffic.

Another issue with the AI traffic vehicles is that they become too sparse after a short period. A future addition could be dynamically generating vehicles near the driver, while discarding vehicles that have driven too far away. A possible implementation of this could use the moving window idea proposed by Olstam [29] and Olstam et al. [30].

5.3.3 Traffic lights. The traffic light system uses a common timing sequence across all junctions in a city. This has the downside that traffic lights are not dynamic and cannot react to traffic conditions, which they often can do in real life. In the future, all the traffic lights at the same junction could be linked and independent of other junctions. Felez et al. [11] describes how the timings of traffic lights at a junction can be synchronised.

5.3.4 Pedestrians. Pedestrians in the simulator can only walk on the pavement around the same block. A more realistic pedestrian model would utilise pedestrian crossings on the road to allow for a more natural flow of foot traffic. They could also have more variance in their movement, such as a running animation or pausing to check their phone. Pedestrians could also benefit from a similar dynamic system proposed in section 5.3.2, where they are generated and discarded based on proximity to the driver.

5.3.5 Motorway improvements. The motorway system currently lacks AI traffic and only generates a limited stretch of road. Section 3.5.1 goes into depth about how the motorway currently works and how it could be improved.

5.3.6 Different vehicle types. Future studies using this driving simulator could look at the differences between driving different types of vehicle. Electric vehicles, in particular, are the future of transportation, and they can be easily simulated in this environment. Other use cases include training emergency service drivers, or new learners who could risk damaging a real car.

5.3.7 Cyclists. The simulator could also be used for analysing the interactions between motorists and cyclists on the road. A cyclist model could be adapted to use the current driver model, allowing it to navigate the environment similarly to a car, and the AI traffic could react to the cyclist accordingly. New highway code rules could be taught to drivers using the simulator.

5.3.8 Motion platform. The current simulator cockpit is stationary, but it could be improved in the future by mounting it to a motion platform, such as the Next Level Racing Traction Plus Platform [28]. This adds a full range of motion to the cockpit, providing drivers with a sense of exactly what the vehicle is doing. An SDK is also available to integrate the motion platform with customised VR projects like this one. However, the driving simulator used by Ropelato et al. [33] featured a motion platform which was found to be inadequate to reduce the disparity between the physical perception and the visual representation of the simulator.

5.3.9 Weather and time. Changing the time of day or adding weather effects like fog, rain, or snow could increase the opportunities for creating different road scenarios. For example, a nighttime mode could be implemented by changing the lighting to come from street lamps and headlights instead of the sun.

5.3.10 GPS navigation. The inside of the car could be fitted with a GPS navigation system. This could be used to study different GPS systems or driver's interactions with them.

5.3.11 Eye tracking. With the ongoing development of virtual reality hardware, some headsets provide an interface to track the eye movement of users. When presenting drivers with various driving scenarios, data on eye movement could be a valuable source of information. However, it would be important not to compromise on the visual fidelity of the simulator, which could risk lower levels of immersion. On the other hand, some headsets use eye tracking for foveated rendering, which could increase the visual fidelity by focusing the rendering at the user's gaze point and placing lower importance on objects in their peripheral vision.

5.3.12 Additional steering wheel controls. The Thrustmaster T300RS GT steering wheel used for CarDriVR lacks control stalks for controls like indicating and lights from real cars. In the future, the control interface could be made more realistic by using a wheel stalk accessory that attaches to the cockpit and provides more realistic access to these controls.

6 CONCLUSION

This project successfully designed and created CarDriVR, a new cost-effective driving simulator for virtual reality with an emphasis on realism and modularity. The simulator allows for the creation of a wide range of virtual driving scenarios in life-sized, realistic cities with detailed road traffic and pedestrians. CarDriVR serves as a versatile tool which can be utilised to study driving behaviours in both controlled and autonomous vehicle situations, with a high degree of realism.

6.1 Word Count

Final word count: 8173

BIBLIOGRAPHY

- [1] Artz, A., Cathey, L., Grant, P., Greenberg, J. and Mariani, M., 2000. The design and construction of the visual subsystem for virtex, the driving simulator at the ford research laboratories.
- [2] Bruck, L., Haycock, B. and Emadi, A., 2021. A review of driving simulation technology and applications. *Ieee open journal of vehicular technology* [Online], 2, pp.1–16. Available from: <https://doi.org/10.1109/OJVT.2020.3036582>.
- [3] Cao, S., Nandakumar, K., Babu, R. and Thompson, B., 2020. Game play in virtual reality driving simulation involving head-mounted display and comparison to desktop display. *Virtual reality* [Online], 24(3), pp.503–513. Available from: <https://doi.org/10.1007/s10055-019-00412-x>.
- [4] Christodoulou, S., Michael-Grigoriou, D., Gregoriades, A. and Pampaka, M., 2013. Design of a 3d interactive simulator for driver behavior analysis. vol. 45.
- [5] Coding, N., 2023. Nwh vehicle physics 2 [Online]. Available from: <http://nwhvehiclephysics.com/doku.php/index>.
- [6] Coding, N., 2023. Nwh vehicle physics 2 change log [Online]. Available from: <http://nwhvehiclephysics.com/doku.php/Changelog>.
- [7] de Frutos, S.H. and Castro, M., 2021. Assessing sim racing software for low-cost driving simulator to road geometric research. *Transportation research procedia* [Online], 58, pp.575–582. XIV Conference on Transport Engineering, CIT2021. Available from: <https://doi.org/https://doi.org/10.1016/j.trpro.2021.11.076>.
- [8] Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A. and Koltun, V., 2017. Carla: An open urban driving simulator [Online]. Available from: <https://proceedings.mlr.press/v78/dosovitskiy17a.html>.
- [9] Drosdol, J. and Panik, F., 1985. The daimler-benz driving simulator a tool for vehicle development. *Sae transactions* [Online], 94, pp.981–997. Available from: <http://www.jstor.org/stable/44467637>.
- [10] Fanatec, 2023. Fanatec [Online]. Available from: <https://fanatec.com/eu-en/>.

- [11] Felez, J., Maroto, J., Cabanellas, J.M. and Mera, J.M., 2013. A full-scale simulation model to reproduce urban traffic in real conditions in driving simulators. *Simulation* [Online], 89(9), sep, p.1099–1114. Available from: <https://doi.org/10.1177/0037549713483557>.
- [12] Game Dev Guide, 2019. Building a traffic system in unity - youtube [Online]. Available from: <https://youtu.be/MXCZ-n5VyJc>.
- [13] Heydinger, G.J., Salaani, M.K., Garrott, W.R. and Grygier, P.A., 2002. Vehicle dynamics modelling for the national advanced driving simulator. *Proceedings of the institution of mechanical engineers, part d: Journal of automobile engineering* [Online], 216(4), pp.307–318. <https://doi.org/10.1243/0954407021529138>, Available from: <https://doi.org/10.1243/0954407021529138>.
- [14] Ihemedu-Steinke, Q., Sirim, D., Erbach, R., Halady, P. and Meixner, G., 2015. Development and evaluation of a virtual reality driving simulator.
- [15] Ihemedu-Steinke, Q., Thull, S., Weber, M., Erbach, R., Meixner, G. and Marsden, N., 2017. Simulation sickness related to virtual reality driving simulation. pp.521–532. Available from: https://doi.org/10.1007/978-3-319-57987-0_42.
- [16] Jovan, 2022. What is force feedback in a racing wheel? [Online]. Available from: <https://kommandotech.com/guides/what-is-force-feedback-in-a-racing-wheel/>.
- [17] kasiopy, 2017. City beach 3d model [Online]. Available from: <https://www.turbosquid.com/3d-models/city-beach-3d-model-1166663>.
- [18] kasiopy, 2018. 3d model kc metropolis [Online]. Available from: <https://www.turbosquid.com/3d-models/3d-model-kc-metropolis-1338278>.
- [19] kasiopy, 2019. City buildings 3d model [Online]. Available from: <https://www.turbosquid.com/3d-models/city-buildings-3d-model-1393069>.
- [20] kasiopy, 2020. 3d town houses buildings [Online]. Available from: <https://www.turbosquid.com/3d-models/3d-town-houses-buildings-1572467>.
- [21] Kemeny, A., 2014. From driving simulation to virtual reality [Online]. Available from: <https://doi.org/10.1145/2617841.2620721>.
- [22] Kugler, L., 2021. The state of virtual reality hardware. *Commun. acm* [Online], 64(2), jan, p.15–16. Available from: <https://doi.org/10.1145/3441290>.
- [23] Lenk, A., 2022. Stylised vehicles pack | unity asset store [Online]. Available from: <https://assetstore.unity.com/packages/3d/vehicles/land/stylized-vehicles-pack-low-poly-121593>.
- [24] Logitech, 2023. Logitech product search - racing [Online]. Available from: <https://www.logitech.com/en-gb/search.html?racing&st=1&start=0&sort=new¤tTab=products>.
- [25] Mohajer, N., Abdi, H., Nelson, K. and Nahavandi, S., 2015. Vehicle motion simulators, a key step towards road vehicle dynamics improvement. *Vehicle system dynamics* [Online], 53(8), pp.1204–1226. <https://doi.org/10.1080/00423114.2015.1039551>, Available from: <https://doi.org/10.1080/00423114.2015.1039551>.
- [26] Mourant, R.R. and Sadhu, P., 2002. Evaluation of force feedback steering in a fixed based driving simulator. *Proceedings of the human factors and ergonomics society annual meeting* [Online], 46(26), pp.2202–2205. Available from: <https://doi.org/10.1177/154193120204602621>.
- [27] Next Level Racing, 2020. Next level racing gttrack [Online]. Available from: <https://nextlevelracing.com/products/next-level-racing-gttrack/>.
- [28] Next Level Racing, 2020. Next level racing traction plus platform [Online]. Available from: <https://nextlevelracing.com/products/traction-plus-platform/>.
- [29] Olstam, J., 2006. Simulation of vehicles in a driving simulator using microscopic traffic simulation [Online]. Available from: <https://doi.org/10.1201/9781439808016-c3>.
- [30] Olstam, J.J., Lundgren, J., Adlers, M. and Matstoms, P., 2008. A framework for simulation of surrounding vehicles in driving simulators. *Acm trans. model. comput. simul.* [Online], 18(3), jul. Available from: <https://doi.org/10.1145/1371574.1371575>.
- [31] Panerai, F., Droulez, J., Kemeny, A., Balligand, E., Favre, B. and Berthelot, M., 2001. Speed and safety distance control in truck driving: comparison of simulation and real-world environment.
- [32] Rebenitsch, L. and Owen, C., 2016. Review on cybersickness in applications and visual displays. *Virtual reality* [Online], 20(2), pp.101–125. Available from: <https://doi.org/10.1007/s10055-016-0285-9>.
- [33] Ropelato, S., Zünd, F., Menozzi, M., Sumner, R. and Magnenat, S., 2017. *Adaptive tutoring on a virtual reality driving simulator*.
- [34] The highway code, 2015. The highway code - light signals. Available from: <https://www.gov.uk/guidance/the-highway-code/light-signals-controlling-traffic>.
- [35] The highway code, 2015. The highway code - stopping distances - rule 126. Available from: <https://www.gov.uk/guidance/the-highway-code/general-rules-techniques-and-advice-for-all-drivers-and-riders-103-to-158>.
- [36] Thrustmaster, 2023. Sim racing wheels, pedal sets and accessories | thrustmaster [Online]. Available from: <https://www.thrustmaster.com/en-ie/universe/racing/>.
- [37] Tudor, E.S., 2020. The emergence of esport during covid-19: how sim racing replaced live motorsport in 2020. *Journal of motorsport culture & history*, 1(1), p.8.
- [38] UMA Steering Group, 2021. Uma 2 - unity multipurpose avatar | unity asset store [Online]. Available from: <https://assetstore.unity.com/packages/3d/characters/uma-2-unity-multipurpose-avatar-35611>.
- [39] Unity, 2021. Navigation system in unity [Online]. Available from: <https://docs.unity3d.com/Manual/nav-NavMeshAgent.html>.
- [40] Unity, 2021. Unity - scripting api: Navmeshagent [Online]. Available from: <https://docs.unity3d.com/ScriptReference/AI.NavMeshAgent.html>.
- [41] Unity, 2022. Navmeshsurface | ai navigation [Online]. Available from: <https://docs.unity.cn/Packages/com.unity.ai.navigation@1.1/manual/NavMeshSurface.html>.
- [42] Valve, 2022. Headset - valve index® - upgrade your experience - valve corporation [Online]. Web page. Valve Corporation. Available from: <https://www.valvesoftware.com/en/index/headset>.
- [43] VIS Games, 2012. Highway construction set | unity asset store [Online]. Available from: <https://assetstore.unity.com/packages/3d/environments/urban/highway-construction-set-4246>.
- [44] Zhao, C., Kim, A.S., Beams, R. and Badano, A., 2022. Spatiotemporal image quality of virtual reality head mounted displays. *Scientific reports* [Online], 12(1), p.20235. Available from: <https://doi.org/10.1038/s41598-022-24345-9>.